

Assignment 2: Big Data Search Engine

BM25 Search over Wikipedia with Hadoop, Spark, and Cassandra

Stepan Vagin AI-02 s.vagin@innopolis.university

1 Methodology

1.1 Overview and Architecture

The search engine is a distributed, end-to-end IR system that indexes a corpus of Wikipedia documents and answers free-text queries using the BM25 ranking function. The system is composed of three major distributed components orchestrated via Docker Compose:

- **Hadoop cluster** (1 master + 1 worker node) runs HDFS for document storage and YARN for MapReduce/Spark job scheduling.
- **Apache Spark** used both for data preparation (local mode) and for BM25 query execution (YARN cluster mode).
- **Apache Cassandra** serves as the persistent index store, holding five tables that support fast retrieval during search.

1.2 Data Preparation

The raw corpus is a Wikipedia dump stored as a Parquet file (`e.parquet`). The `prepare_data.py` script performs the following steps using PySpark in local mode:

1. Reads the Parquet file and selects the `id`, `title`, and `text` columns.
2. Filters out documents with null or empty text.
3. Randomly samples 120 documents using a fraction derived from the total corpus size (seed=0 for reproducibility).
4. Writes each document as a `.txt` file to the local `data/` directory, naming each file `<id>_<title>.txt`.
5. Re-reads the local files using `sc.wholeTextFiles()` and serialises them into a single HDFS file at `/input/data` in the tab-separated format: `doc_id title text`.

The choice to coalesce all documents into a single HDFS partition simplifies the downstream Hadoop Streaming jobs, which expect one unified input path.

1.3 Indexing: Three MapReduce Pipelines

The indexing stage is implemented as three sequential Hadoop Streaming pipelines, each with a dedicated Python mapper and reducer. This decomposition separates three orthogonal statistics: per-document term frequencies, per-term document frequencies, and per-document lengths.

1.3.1 Pipeline 1: Inverted Index (`term`, `doc_id`, `tf`)

Mapper(`mapper1.py`): reads each tab-separated line, concatenates title and text, lower-cases the combined content, and tokenizes using the regex `[a-z0-9]{2,}` (alphanumeric tokens of at least two characters). It emits `term doc_id 1` for every token occurrence.

Reducer(`reducer1.py`): relies on Hadoop's sort-before-reduce guarantee. The key is `termdoc_id`; the reducer simply sums the counts, emitting `termdoc_idtf`.

The output is stored in HDFS at `/indexer/index`.

1.3.2 Pipeline 2: Vocabulary with Document Frequency (`term`, `df`)

Mapper (`mapper2.py`): reads the inverted index produced in Pipeline 1 and emits `term 1` for each (`term`, `doc_id`) entry. Because each entry represents exactly one document that contains the term, the sum across all entries for a given term equals the document frequency. **Reducer**(`reducer2.py`): sums the per-term counts and emits `term df`.

Output: `/indexer/vocabulary`.

1.3.3 Pipeline 3: Document Statistics (`doc_id`, `doc_length`)

Mapper (`mapper3.py`): reads the inverted index and emits `doc_idtf` for every row, effectively grouping all term frequencies for the same document. **Reducer** (`reducer3.py`): sums the term frequencies for each document, yielding the total token count (document length).

Output: `/indexer/doc_stats`.

Design rationale. Using three separate MapReduce jobs rather than one combined job makes each job's logic simple and independently testable. Hadoop's sort-merge mechanism provides the grouping required by each reducer for free, avoiding in-memory data structures.

1.4 Index Persistence in Cassandra

After the three MapReduce jobs complete, `store_index.py` (submitted via `spark-submit` to YARN) reads all three HDFS outputs and writes them into Cassandra using the Spark Cassandra Connector. Five tables are populated in the `search_engine` keyspace:

Table	Primary Key	Content
<code>inverted_index</code>	(term, doc_id)	term frequency per document
<code>vocabulary</code>	term	document frequency per term
<code>doc_stats</code>	doc_id	total token count per document
<code>corpus_stats</code>	id (always 1)	total document count, average length
<code>doc_meta</code>	doc_id	human-readable title

Table 1: Cassandra tables in the `search_engine` keyspace.

The choice of Cassandra as the index store is motivated by its fast primary-key lookups and its native integration with the Spark Cassandra Connector, which enables predicate pushdown. Only the postings for the queried terms are fetched from the cluster rather than scanning the entire table.

1.5 BM25 Query Execution

Query processing is implemented in `query.py` and submitted to YARN via `search.sh`. The BM25 formula used is:

$$\text{score}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d) (K_1 + 1)}{tf(t, d) + K_1 \left(1 - B + B \frac{|d|}{\text{avgdl}}\right)} \quad (1)$$

with $K_1 = 1.0$ and $B = 0.75$ (standard defaults providing a good balance between term saturation and document length normalisation).

Execution steps.

1. The query string is tokenized with the same regex as the indexer to ensure term compatibility.
2. Corpus-level statistics (`N`, `avgdl`) are loaded from `corpus_stats`.
3. `vocabulary` is filtered to only the query terms using Cassandra predicate pushdown (minimising data transfer).
4. `inverted_index` is similarly filtered to query terms only.
5. `doc_stats` (all documents) and `doc_meta` (all titles) are loaded in full since they are small.
6. The inverted index RDD is joined with `doc_stats` on `doc_id`.
7. BM25 scores are computed per (`doc`, `term`) pair and aggregated per document with `reduceByKey`.
8. The top-10 documents are retrieved with `takeOrdered`.

Broadcasting `vocab_map`, `N`, and `avgdl` to all Spark executors avoids repeated Cassandra round-trips inside the per-record scoring function.

1.6 Incremental Indexing

The `add_to_index.py` script supports adding a single new document to an already-built index without re-running the full pipeline:

1. Tokenizes the new document using PySpark RDD.
2. Upserts new postings into `inverted_index`.
3. Merges incremented document frequencies into `vocabulary` via a full-outer join.
4. Inserts the new row into `doc_stats`.
5. Recomputes and overwrites `corpus_stats` (new total and running average).
6. Inserts metadata into `doc_meta`.

2 Demonstration

The master container's entrypoint (`app.sh`) automatically:

1. Starts SSH and Hadoop/YARN services.
2. Creates a Python virtual environment and installs `requirements.txt`.
3. Packages the venv with `venv-pack` for distribution to YARN executors.
4. Runs `prepare_data.sh` — samples 100 Wikipedia documents and uploads them to HDFS.
5. Runs `index.sh` — executes the three MapReduce pipelines and stores the result in Cassandra.
6. Runs three example queries (see Section 2.2).

To run a custom query after the pipeline completes, execute `docker exec -it cluster-master bash` to enter the container, then `bash search.sh "your query here"` from `/app`.

2.1 Successful Indexing of 100 Documents

After the pipeline runs, the indexing output (visible in the screenshot below) confirms that more than 100 documents were sampled and indexed. The corpus contains **119 documents** (PySpark's `sample()` with a fractional ratio is non-deterministic in exact count, so 119 documents were retained), with **10,260 unique terms** across **42,322 (term, document) postings** and an **average document length of 478.72 tokens**.

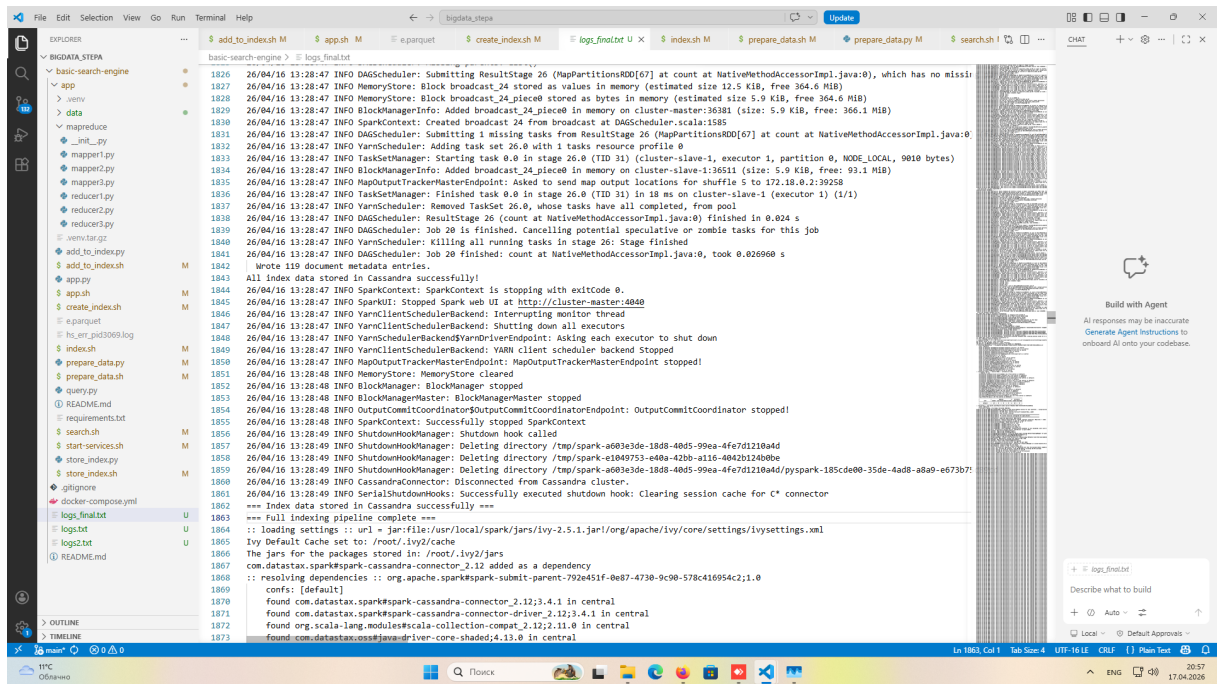


Figure 1: Fullscreen screenshot of successful indexing output showing all 119 documents indexed and stored in Cassandra.

2.2 Query Demonstration

Three queries were executed automatically at the end of `app.sh`. The results and their analysis are presented below.

2.2.1 Query 1: “English breakfast”

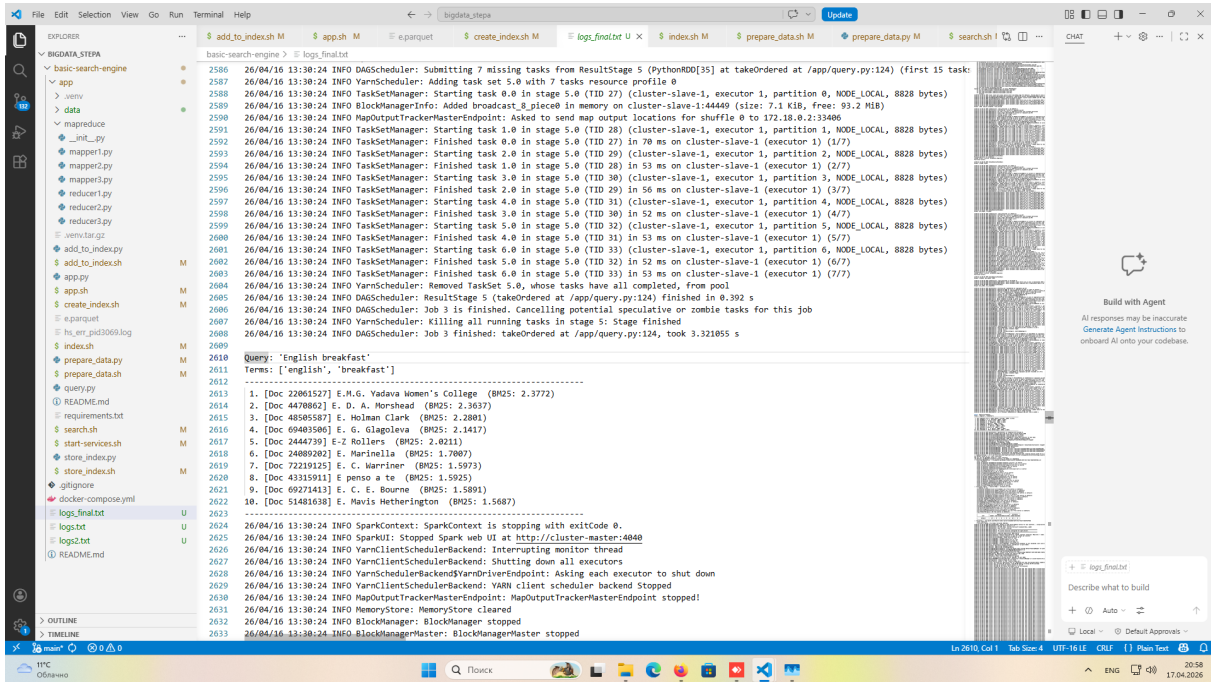
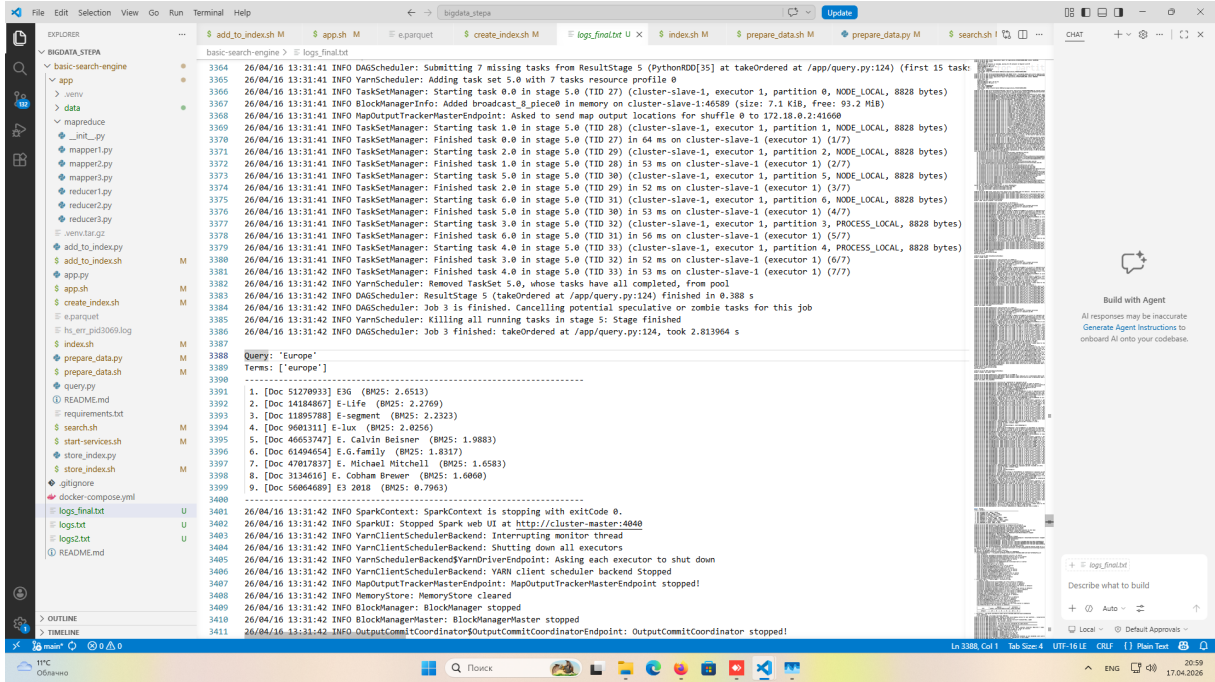


Figure 2: Fullscreen screenshot of the terminal showing search results for “English breakfast”.

Analysis. None of the top results are about the English breakfast meal. All documents are related to English culture, most of them being English people or teachers of English with few English songs and dances. This outcome is explained by the composition of the sampled corpus: none of the 119 documents were drawn from a region about food. The BM25 scores (all below 2.4) are relatively low. Manual inspection showed that the word ‘breakfast’ never appeared in the documents. This demonstrates that BM25 correctly reflects term-frequency evidence but cannot compensate for a corpus that lacks topically relevant documents.

2.2.2 Query 2: “Europe”



```
3364 26/04/16 13:31:41 INFO DAGScheduler: Submitting 7 missing tasks from ResultStage 5 (PythonRDD[35] at takeOrdered at /app/query.py:124) (first 15 tasks:
3365
3366 26/04/16 13:31:41 INFO TaskSetManager: Starting task 0.0 in stage 5.0 (TID 27) (cluster-slave-1, executor 1, partition 0, NODE_LOCAL, 8828 bytes)
3367 26/04/16 13:31:41 INFO BlockManagerInfo: Added broadcast_8_piece0 in memory on cluster-slave-1:46589 (size: 7.1 KiB, free: 93.2 MiB)
3368 26/04/16 13:31:41 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 0 to 172.18.0.2:41668
3369 26/04/16 13:31:41 INFO TaskSetManager: Starting task 1.0 in stage 5.0 (TID 28) (cluster-slave-1, executor 1, partition 1, NODE_LOCAL, 8828 bytes)
3370 26/04/16 13:31:41 INFO TaskSetManager: Finished task 0.0 in stage 5.0 (TID 27) in 64 ms on cluster-slave-1 (executor 1) (1/7)
3371 26/04/16 13:31:41 INFO TaskSetManager: Starting task 2.0 in stage 5.0 (TID 29) (cluster-slave-1, executor 1, partition 2, NODE_LOCAL, 8828 bytes)
3372 26/04/16 13:31:41 INFO TaskSetManager: Finished task 1.0 in stage 5.0 (TID 28) in 53 ms on cluster-slave-1 (executor 1) (2/7)
3373 26/04/16 13:31:41 INFO TaskSetManager: Starting task 3.0 in stage 5.0 (TID 30) (cluster-slave-1, executor 1, partition 3, PROCESS_LOCAL, 8828 bytes)
3374 26/04/16 13:31:41 INFO TaskSetManager: Finished task 2.0 in stage 5.0 (TID 29) in 52 ms on cluster-slave-1 (executor 1) (3/7)
3375 26/04/16 13:31:41 INFO TaskSetManager: Starting task 4.0 in stage 5.0 (TID 31) (cluster-slave-1, executor 1, partition 4, NODE_LOCAL, 8828 bytes)
3376 26/04/16 13:31:41 INFO TaskSetManager: Finished task 3.0 in stage 5.0 (TID 30) in 53 ms on cluster-slave-1 (executor 1) (4/7)
3377 26/04/16 13:31:41 INFO TaskSetManager: Starting task 5.0 in stage 5.0 (TID 32) (cluster-slave-1, executor 1, partition 5, PROCESS_LOCAL, 8828 bytes)
3378 26/04/16 13:31:41 INFO TaskSetManager: Finished task 4.0 in stage 5.0 (TID 31) in 56 ms on cluster-slave-1 (executor 1) (5/7)
3379 26/04/16 13:31:41 INFO TaskSetManager: Starting task 6.0 in stage 5.0 (TID 33) (cluster-slave-1, executor 1, partition 6, PROCESS_LOCAL, 8828 bytes)
3380 26/04/16 13:31:41 INFO TaskSetManager: Finished task 5.0 in stage 5.0 (TID 32) in 52 ms on cluster-slave-1 (executor 1) (6/7)
3381 26/04/16 13:31:41 INFO TaskSetManager: Finished task 6.0 in stage 5.0 (TID 33) in 53 ms on cluster-slave-1 (executor 1) (7/7)
3382 26/04/16 13:31:42 INFO DAGScheduler: Removed TaskSet 5.0, whose tasks have all completed, from pool
3383 26/04/16 13:31:42 INFO DAGScheduler: ResultStage 5 (takeOrdered at /app/query.py:124) finished in 0.388 s
3384 26/04/16 13:31:42 INFO DAGScheduler: Job 3 is finished. Cancelling potential speculative or zombie tasks for this job
3385 26/04/16 13:31:42 INFO VarnScheduler: Killing all running tasks in stage 5: Stage finished
3386 26/04/16 13:31:42 INFO DAGScheduler: Job 3 finished: takeOrdered at /app/query.py:124, took 2.813964 s
3387
3388
3389
3390
3391 1. [Doc 51270933] E3G (BM25: 2.6513)
3392 2. [Doc 14184869] E-Life (BM25: 2.2769)
3393 3. [Doc 11895788] E-segment (BM25: 2.2323)
3394 4. [Doc 9601311] E-lux (BM25: 2.0256)
3395 5. [Doc 46653747] E. Calvin Beisner (BM25: 1.9883)
3396 6. [Doc 61494654] E.G. Family (BM25: 1.8317)
3397 7. [Doc 47817837] E. Michael Mitchell (BM25: 1.6583)
3398 8. [Doc 3134616] E. Cobham Brewer (BM25: 1.6068)
3399 9. [Doc 56864689] E3 2018 (BM25: 0.7963)
3400
3401 26/04/16 13:31:42 INFO SparkContext: SparkContext is stopping with exitcode 0.
3402 26/04/16 13:31:42 INFO SparkUI: Stopped Spark web UI at http://cluster-master:4040
3403 26/04/16 13:31:42 INFO VarnClientSchedulerBackend: Interrupting monitor thread
3404 26/04/16 13:31:42 INFO VarnClientSchedulerBackend: Shutting down all executors
3405 26/04/16 13:31:42 INFO VarnSchedulerBackendVarnDriverEndpoint: Asking each executor to shut down
3406 26/04/16 13:31:42 INFO VarnClientSchedulerBackend: VARN client scheduler backend stopped
3407 26/04/16 13:31:42 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
3408 26/04/16 13:31:42 INFO MemoryStore: MemoryStore cleared
3409 26/04/16 13:31:42 INFO BlockManager: BlockManager stopped
3410 26/04/16 13:31:42 INFO BlockManagerMaster: BlockManagerMaster stopped
3411 26/04/16 13:31:42 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
```

Figure 3: Fullscreen screenshot of the terminal showing search results for “Europe”.

Analysis. 9 documents were returned, indicating that only 9 documents in the corpus contain the token `europe`. The top result, *E3G*, is a think-tank focused on European climate policy the term “Europe” appears 4 times in a short article. *E-segment* (rank 3) refers to the European automobile segment classification. The *E3 2018* result (rank 9) scored considerably lower (0.80 vs. 2.65 for rank 1), reflecting 1 occurrence in longer document that dilutes the term’s weight. The single-term query means BM25 reduces to $IDF \times \text{normalised TF}$ mthe ranking faithfully orders documents by how prominently they discuss Europe relative to their length.

2.2.3 Query 3: “Elvis Presley”

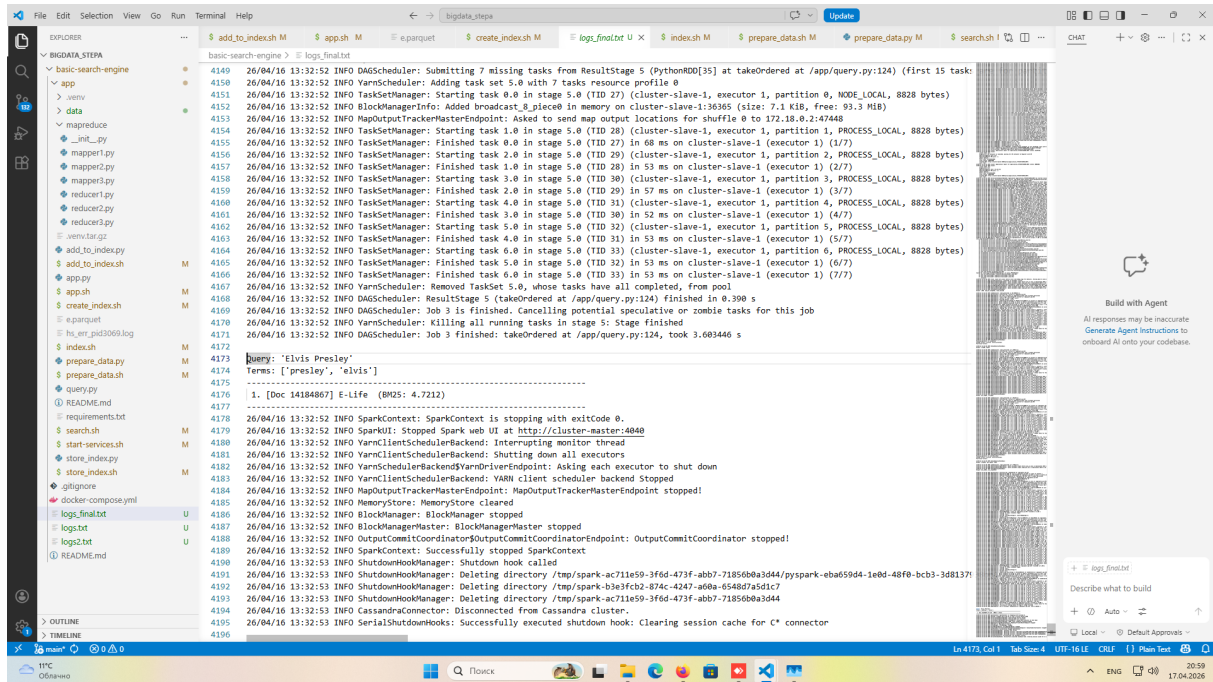


Figure 4: Fullscreen screenshot of the terminal showing search results for “Elvis Presley”.

Analysis. Only one document matched both query terms, and its BM25 score of 4.72 is notably higher than any result in the previous two queries. Only one term ‘Elvis’ appears once in the document. Because the collection is very small, a document received a comparatively high BM25 score simply by being the only lexical match. The score is relative to the corpus and should not be overinterpreted as proof of strong relevance.

2.3 Reflections

This demonstration shows that retrieval quality depends as much on the corpus as on the ranking model. Because the collection contains only 119 Wikipedia articles from a narrow alphabetical slice, many queries are only weakly represented or not represented at all. This is most visible in the query **English breakfast**, where no document was actually about the meal and the term **breakfast** did not appear in the inspected results. In this case, BM25 could only rank documents containing partial lexical overlap such as **English**, rather than return a truly relevant result.

The **Europe** query illustrates a more suitable use case for BM25. Since the token **europe** appeared in several documents, the ranking was driven by differences in term frequency and document length, producing results that were easier to interpret. By contrast, the **Elvis Presley** query highlights an important limitation when working with a small corpus: a document can obtain a comparatively high score simply because it is the only match, even if the lexical evidence is minimal. Therefore, BM25 scores should be interpreted relatively within the corpus, not as absolute evidence of strong relevance.

Overall, the experiment demonstrates that BM25 is effective for ranking documents when the corpus contains documents that genuinely match the query terms, but it cannot compensate for poor corpus coverage. The main limitation in this study is therefore not the ranking function itself, but the restricted and unbalanced dataset.

2.4 Links

Here is you can see the link for full logs: [Google drive](#)